

Teams & Enterprise

Service Accounts

 Service accounts are available on the [Enterprise plan](#).

Service accounts are non-human accounts that enable teams to securely automate Cursor-powered workflows at scale. With service accounts, you can consume APIs, authenticate the [CLI](#), and invoke [cloud agents](#) without tying integrations to individual developers' personal accounts.

Why use service accounts

As teams find new ways to automate coding tasks with Cursor cloud agents, APIs, and CLI, the need for centralized, secure automation becomes critical. Service accounts address this by:

- **Decoupling from individuals:** Automations continue running even as people and roles change
- **Secure credential management:** Easily rotate API keys without disrupting workflows
- **Centralized access control:** Admins manage all service account permissions in one place
- **Attribution and auditability:** Tie cloud agent runs to the initiating service or system

Key features

No additional seat required

Service accounts are included with your Enterprise plan at no extra cost. They do not consume a seat license.

Usage consumption

Service accounts consume usage from your team's usage pool, just like human users. All usage is tracked and visible in your team's analytics and billing.

Cloud agent integration

Service accounts can initiate [cloud agent](#) runs programmatically. This enables automation scenarios such as:

- A ticket created in Linear triggering a cloud agent to implement a feature
- An error in Sentry initiating a cloud agent to investigate and fix the issue
- Internal engineering services kicking off migrations or refactoring tasks

Admin visibility

Cloud agent runs initiated by service accounts are accessible to all team admins. This ensures visibility and oversight of automated workflows across your organization.

Repository access

Service accounts can initiate cloud agent runs on any repository that has been authorized via the [Cursor GitHub app](#).



The GitHub integration must be connected at the team level for service accounts to access repositories. If you have a personal GitHub integration but no team-level integration, service accounts will not be able to initiate cloud agent runs.

To connect GitHub at the team level:

1. Navigate to **Dashboard** → **Settings** → **Integrations**
2. Connect the Cursor GitHub app to your organization
3. Authorize the repositories you want service accounts to access

Repository access is governed by the permissions configured for your team's GitHub app installation.

Creating a service account

Admins can create and manage service accounts from the [Cursor Dashboard](#).

1. Navigate to **Dashboard** → **Settings** → **Service Accounts**
2. Click **New Service Account**
3. Enter a name and optional description for the service account
4. Click **Create**

When you create a service account, an API key is generated. Copy this key immediately—it will only be shown once and cannot be retrieved later.

 Store your API key securely. If you lose it, you'll need to rotate it to generate a new one.

Managing API keys

Each service account can have API keys associated with it. You can:

- **View masked keys:** See the last few characters of each key for identification
- **Rotate keys:** Generate a new key and invalidate the old one
- **Archive service accounts:** Archive a service account and revoke all its API keys

Rotating an API key

To rotate an API key:

1. Navigate to **Dashboard** → **Settings** → **Service Accounts**
2. Find the service account and click the rotate icon next to its API key
3. Copy the new key immediately

The old key is immediately invalidated. Update any integrations using the old key.

Using service accounts with the API

Service accounts authenticate using their API key. Use the key in the `Authorization` header when making requests to the [Cloud Agents API](#):

```
1 curl -X POST https://api.cursor.com/agents \
2   -H "Authorization: Bearer YOUR_SERVICE_ACCOUNT_API_KEY" \
3   -H "Content-Type: application/json" \
4   -d '{
5     "repo": "owner/repo",
6     "prompt": "Implement the feature described in issue #123"
7   }'
```

See the [Cloud Agents API documentation](#) for the full API reference.

Using service accounts with the CLI

Service accounts can authenticate the [Cursor CLI](#) by setting the API key as `CURSOR_API_KEY`. This is the recommended way to run the CLI in CI/CD pipelines, cron jobs, and other non-interactive environments where browser login isn't possible.

```
1 export CURSOR_API_KEY=your_service_account_api_key
2
3 # Run a task in a CI pipeline
4 agent -p --force "Refactor the authentication module to use OAuth 2.0"
```

The same environment variable works in any context, including local development. See the [CLI authentication docs](#) for all authentication options and the [headless CLI guide](#) for scripting patterns.

Security best practices

- **Rotate keys regularly:** Establish a key rotation schedule for your service accounts
- **Use descriptive names:** Name service accounts after their purpose (e.g., "Linear Integration", "Sentry Auto-Fix")
- **Limit scope:** Create separate service accounts for different automation workflows
- **Monitor usage:** Review service account activity in your team's analytics dashboard
- **Revoke unused accounts:** Archive service accounts that are no longer in use

Archiving a service account

Archiving a service account:

- Revokes all API keys associated with the account
- Breaks any integrations using those keys
- Preserves the account record for auditability

To archive a service account:

1. Navigate to **Dashboard** → **Settings** → **Service Accounts**
2. Click the archive icon next to the service account
3. Confirm the archive action



Archived accounts can be viewed by clicking **Show Archived** on the Service Accounts page. This helps maintain a complete audit trail of service accounts used by your team.

Service accounts are available on the Enterprise plan

Automate Cursor-powered workflows at scale with non-human accounts for APIs and cloud agents.

Contact Sales



English ▾